

ЛАБОРАТОРНАЯ РАБОТА № 2

«Разработка простого REST-сервиса с валидацией»

Студент(ка): Зырянова Татьяна Евгеньевна

Группа: РИМ – 150950

Предмет: Spring Boot

Репозиторий ДЗ: https://github.com/ZyrT12/java-core_2026-2027

Цель: разработать простой REST-сервис.

Описание задачи: описать входящее (request) и исходящее (response) сообщения, представить схему работы сервиса, реализовать REST-сервис на Spring Boot, проверить его через Postman, добавить валидацию данных и обработку ошибок. В дополнительной части необходимо проверить условия валидации по отдельности и вместе, а также реализовать исключение `UnsupportedCodeException` для `uid = 123`. В отчете привести код или скриншоты кода, результаты тестов, ответы на вопросы, ссылку на GitHub и вывод

Ход выполнения работы

1. Описание входящего и исходящего сообщений

1.1. Описание входящего сообщения (request)

Ключ	Тип	Обязательность / ограничение	Описание
uid	String	Да, до 32 символов	Уникальный идентификатор сообщения
operationUid	String	Да, до 32 символов	Уникальный идентификатор операции
systemName	String	Нет	Имя системы отправителя
systemTime	String	Да	Время создания сообщения
source	String	Нет	Наименование ресурса
communicationId	int	От 1 до 100000	Уникальный идентификатор коммуникации
templateId	int	Нет	Уникальный идентификатор шаблона
productCode	int	Нет	Код продукта
smsCode	int	Нет	SMS-код

Пример входящего JSON-сообщения из задания:

```
{
  "uid": "1",
  "operationUid": "",
  "systemName": "",
  "systemTime": "",
  "source": "",
  "communicationId": 100,
  "templateId": 8,
  "productCode": 1500,
  "smsCode": 10
}
```

1.2. Описание ответного сообщения (response)

Ключ	Тип	Обязательность	Возможные значения / примечание
uid	String	Да	Идентификатор сообщения
operationUid	String	Да	Идентификатор операции
systemTime	String	Да	Время/значение из ответа сервиса
code	String	Да	success, failed
errorCode	String	Да	UnsupportedCodeException, ValidationException, UnknownException
errorMessage	String	Да	Сообщение об ошибке

Пример ответного JSON-сообщения из задания:

```
{  
  "code": "success",  
  "errorCode": "",  
  "errorMessage": "",  
  "operationUid": "",  
  "systemTime": "2023-10-07T07:47:56.503Z",  
  "uid": "1"  
}
```

2. Разработка и тестирование REST-сервиса

2.2. Создание пакетов и классов

В проекте создаются пакеты controller, model, exception, service. В пакете model размещаются классы Request и Response.

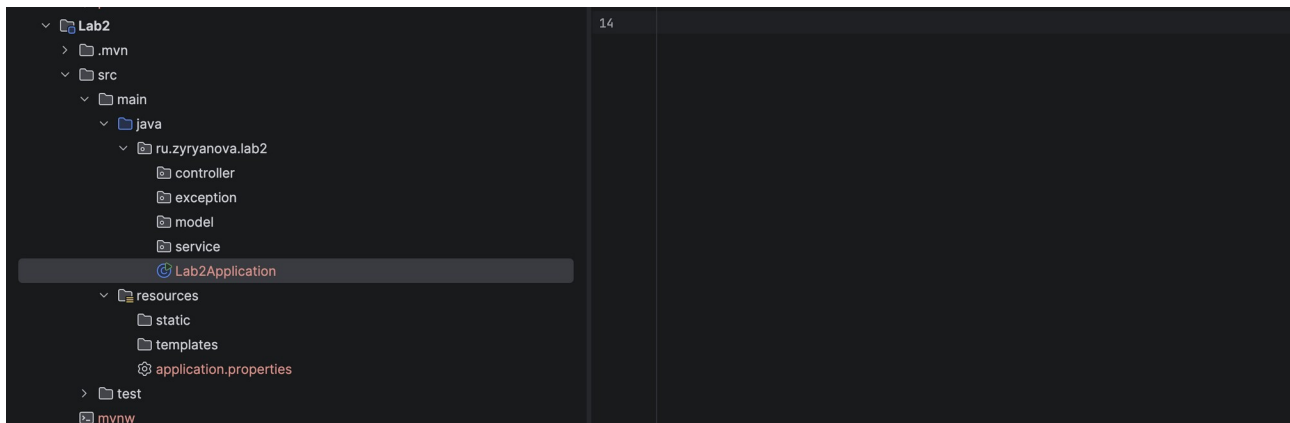


Рисунок 1. Создание пакетов controller, model, exception и service

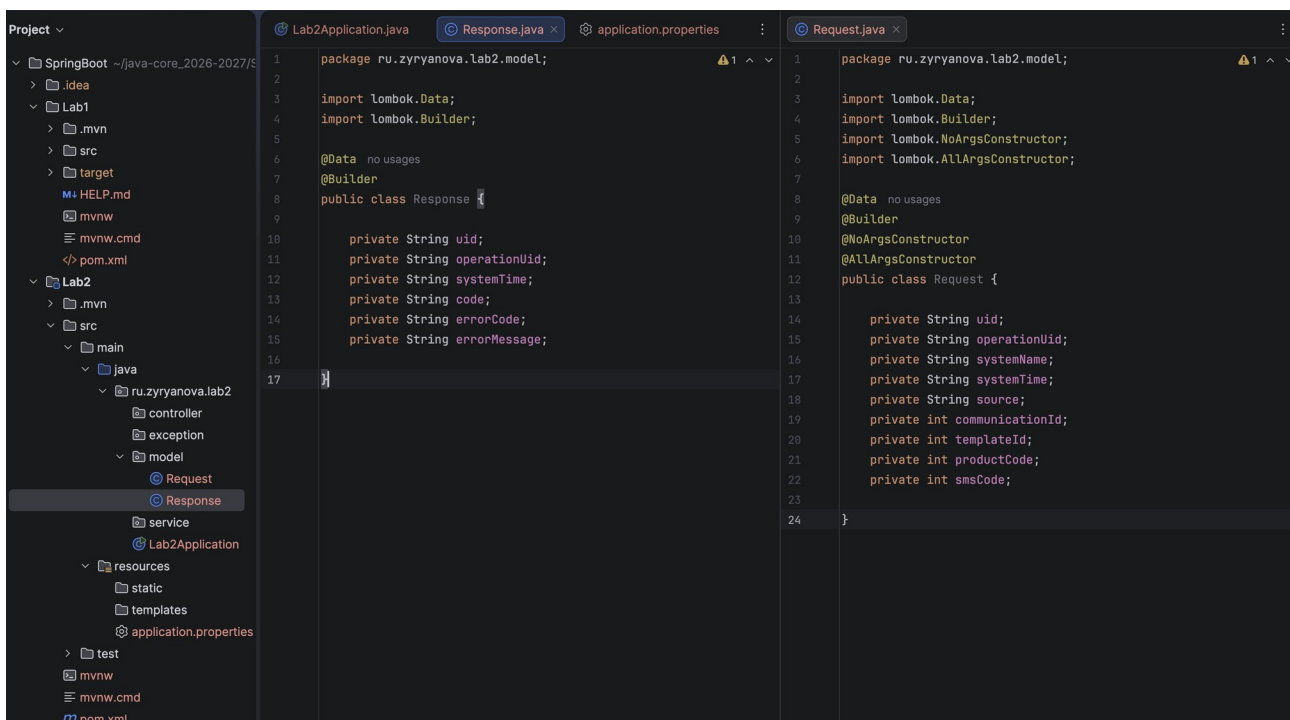


Рисунок 2. Классы Request и Response

2.3. Реализация контроллера и запуск

Создается `MyController`; в `application.properties` устанавливается порт 8082, после чего приложение запускается.

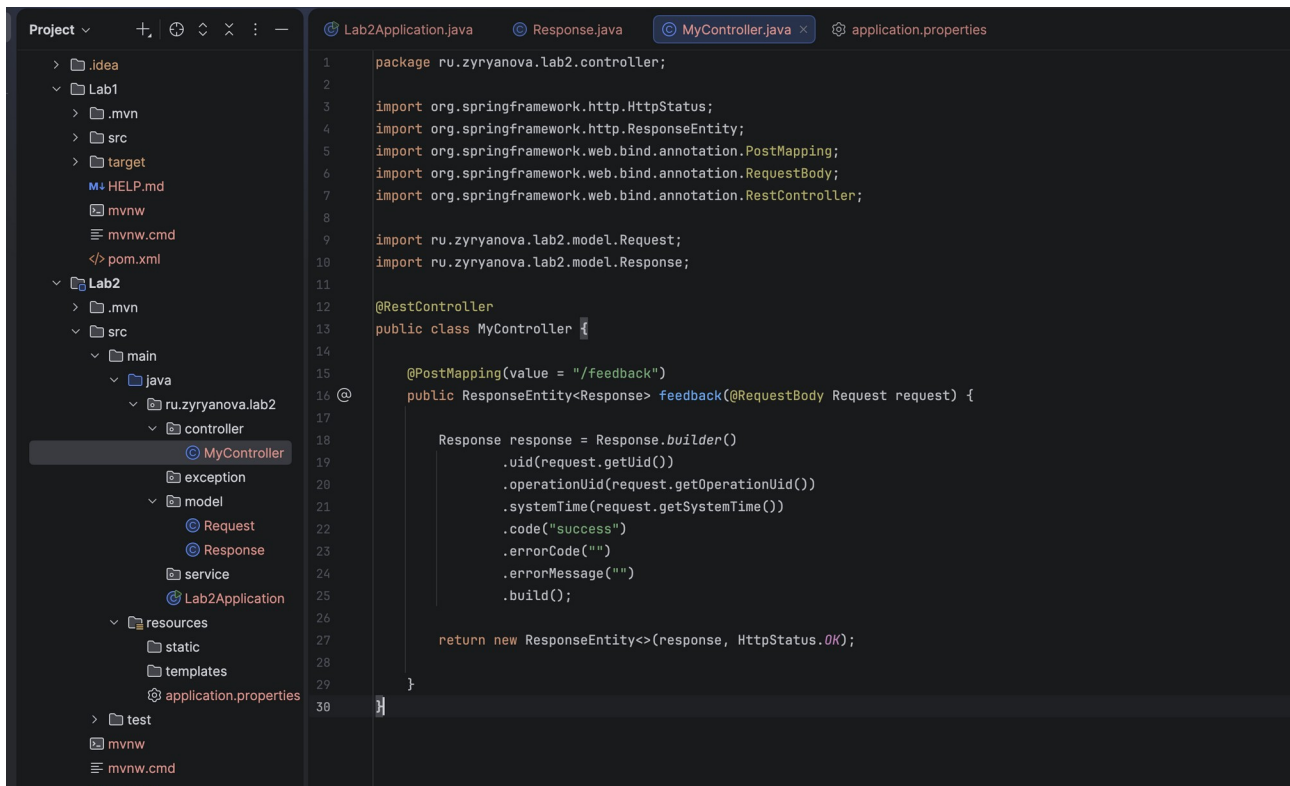


Рисунок 3. Реализация класса `MyController`

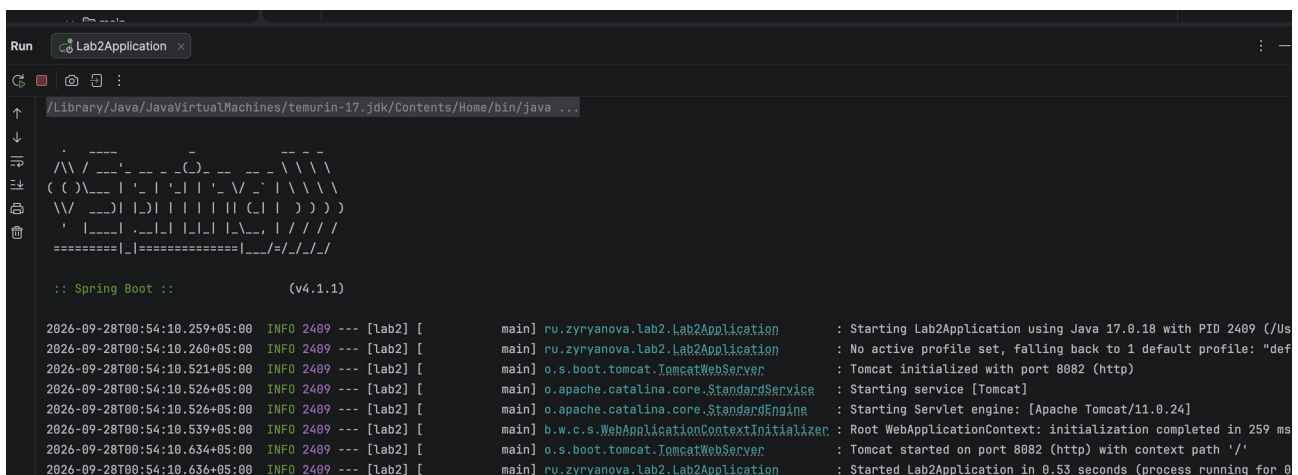


Рисунок 4. Успешный запуск REST-сервиса

2.4. Тестирование REST-сервиса в Postman

В Postman отправляется запрос с JSON-телом и проверяется корректный ответ сервиса.

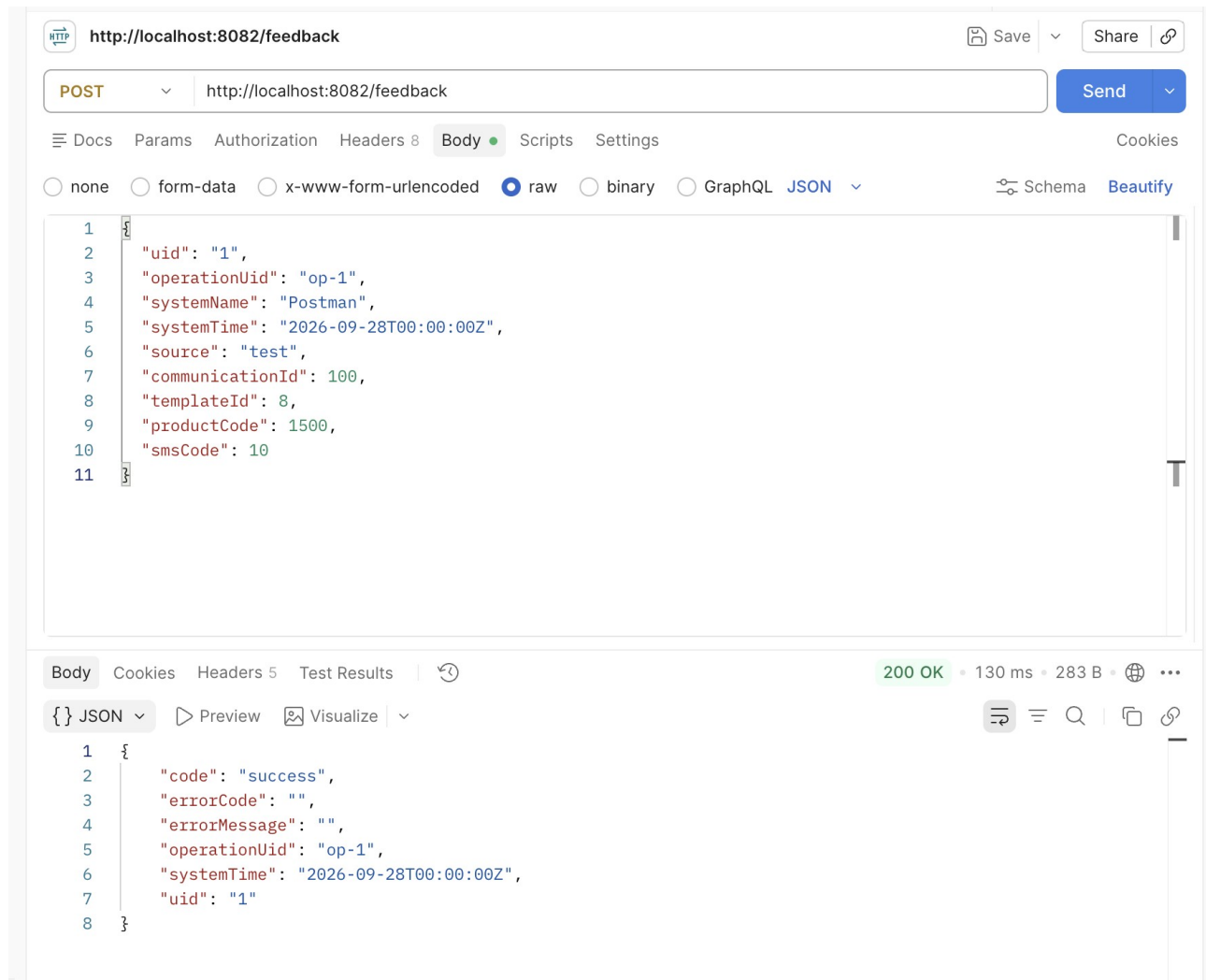


Рисунок 5. Результат тестирования REST-сервиса в Postman

2.5. Добавление валидации и обработка ошибок

В Request добавляется аннотация @NotBlank для uid, создается ValidationFailedException, интерфейс и его реализация в пакете service. Затем дорабатывается MyController.

```
package ru.zyryanova.lab2.model;

import lombok.Data;
import lombok.Builder;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;

import jakarta.validation.constraints.NotBlank;

@Data 2 usages
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Request {

    @NotBlank
    private String uid;

    private String operationUid;
    private String systemName;
    private String systemTime;
    private String source;
    private int communicationId;
    private int templateId;
    private int productCode;
    private int smsCode;

}
```

Рисунок 6. Добавление аннотации @NotBlank к полю uid

```

1 package ru.zyryanova.lab2.exception;
2
3 public class ValidationFailedException extends Exception { no usages
4
5     public ValidationFailedException(String message) { no usages
6         super(message);
7     }
8
9 }

```

Рисунок 7. Создание класса ValidationFailedException

```

package ru.zyryanova.lab2.service;

import org.springframework.stereotype.Service;
import org.springframework.validation.BindingResult;

import ru.zyryanova.lab2.exception.ValidationFailedException;

@Service no usages 1 implementation
public interface ValidationService {

    void isValid(BindingResult bindingResult) no usages 1 implementation
        throws ValidationFailedException;
}

package ru.zyryanova.lab2.service;

import org.springframework.stereotype.Service;
import org.springframework.validation.BindingResult;

import ru.zyryanova.lab2.exception.ValidationFailedException;

@Service
public class RequestValidationService implements ValidationService {

    @Override no usages
    public void isValid(BindingResult bindingResult)
        throws ValidationFailedException {

        if (bindingResult.hasErrors()) {

            throw new ValidationFailedException(
                bindingResult.getFieldError().toString()
            );

        }

    }
}

```

Рисунок 8. Создание интерфейса и реализующего класса в пакете service

```

> .idea
> Lab1
> .mvn
> src
> target
M: HELP.md
mvnw
mvnw.cmd
</pom.xml
> Lab2
> .mvn
> src
> main
> java
> ru
>
> resources
> static
> templates
> test
> target
mvnw

1 package ru.zyryanova.lab2.controller;
2
3 import jakarta.validation.Valid;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.validation.BindingResult;
9 import org.springframework.web.bind.annotation.PostMapping;
10 import org.springframework.web.bind.annotation.RequestBody;
11 import org.springframework.web.bind.annotation.RestController;
12
13 import ru.zyryanova.lab2.model.Request;
14 import ru.zyryanova.lab2.model.Response;
15 import ru.zyryanova.lab2.service.ValidationService;
16 import ru.zyryanova.lab2.exception.ValidationFailedException;
17
18 @RestController
19 public class MyController {
20
21     private final ValidationService validationService; 2 usages
22
23     @Autowired
24     public MyController(ValidationService validationService) {
25         this.validationService = validationService;
26     }
27
28     @PostMapping(value = "/feedback")
29     public ResponseEntity<Response> feedback(
30         @Valid @RequestBody Request request,
31         BindingResult bindingResult) {
32
33         Response response = Response.builder()
34             .uid(request.getUid())
35             .operationUid(request.getOperationUid())
36

```

Рисунок 9. Доработка класса MyController

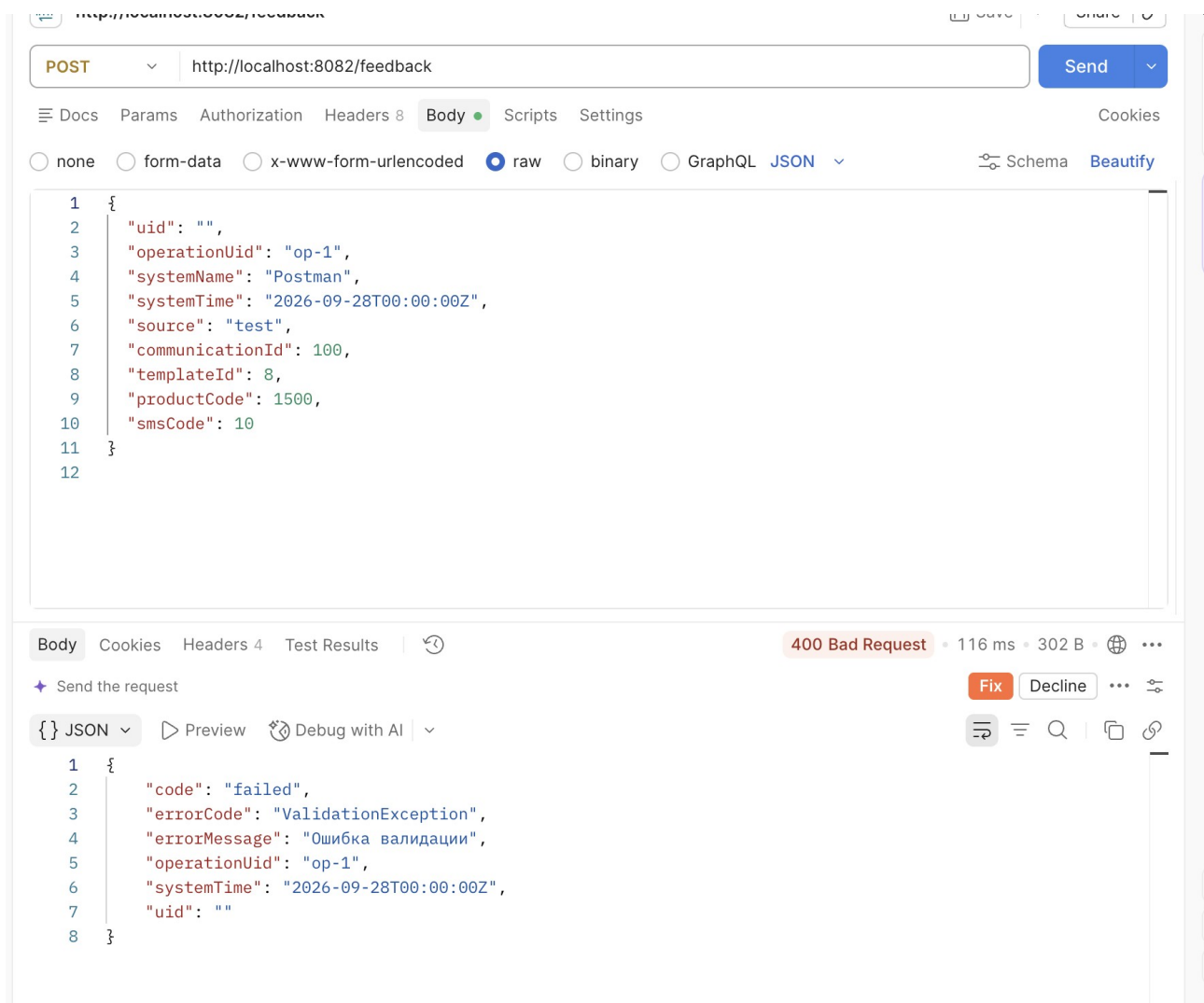


Рисунок 10. Тестирование Postman

3. Реализация дополнительного функционала

3.1. Валидация полей request

В Request добавлены проверки обязательных полей и ограничений по таблице входящего сообщения. В Postman сначала проверяются отдельные некорректные значения, затем несколько ошибок одновременно.

№	Проверяемое условие	Фактический результат / статус
1	uid отсутствует или пустой	400 Bad Request — ошибка валидации
2	uid длиннее 32 символов	400 Bad Request — ошибка валидации
3	operationUid отсутствует или пустой	400 Bad Request — ошибка валидации
4	operationUid длиннее 32 символов	400 Bad Request — ошибка валидации
5	systemTime отсутствует или пустой	400 Bad Request — ошибка валидации
6	communicationId = 0 (меньше 1)	400 Bad Request — ошибка валидации
7	communicationId = 100001 (больше 100000)	400 Bad Request — ошибка валидации
8	Несколько нарушений одновременно	400 Bad Request — ошибка валидации

3.1.1. Проверка пустого значения uid

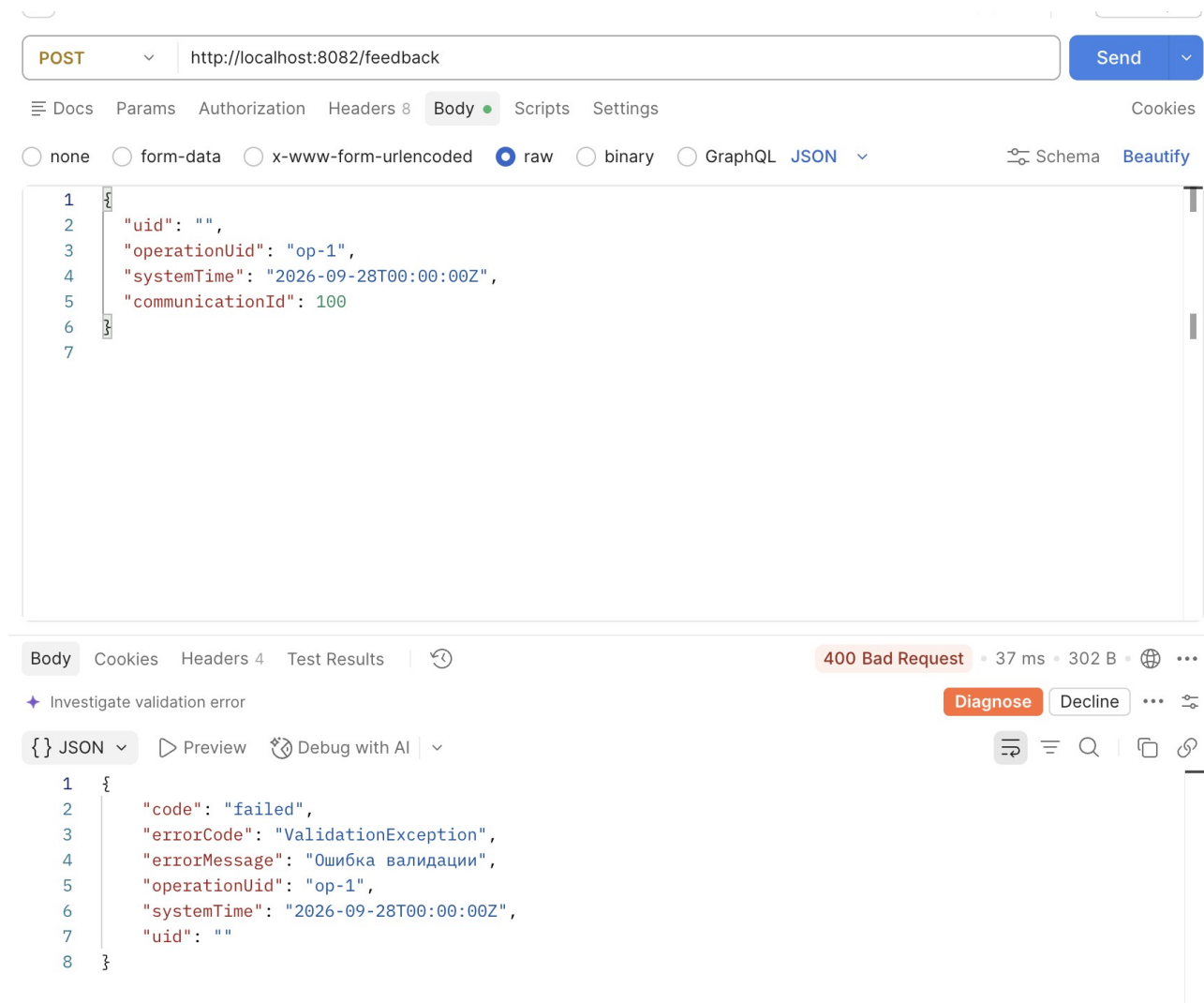


Рисунок 11. Проверка пустого значения uid в Postman

3.1.2. Проверка длины uid более 32 символов

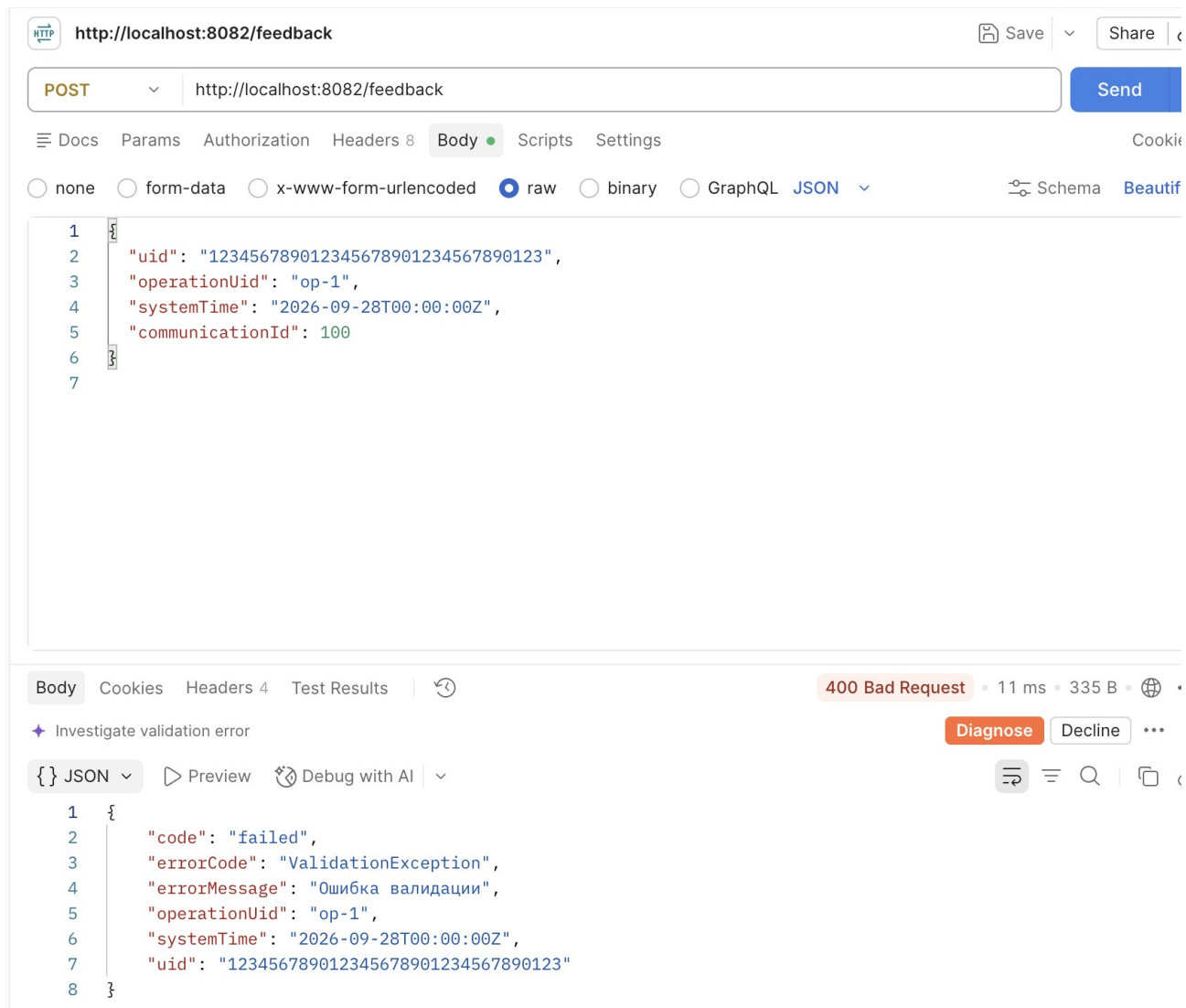


Рисунок 12. Проверка длины uid более 32 символов в Postman

3.1.3. Проверка пустого значения operationUid

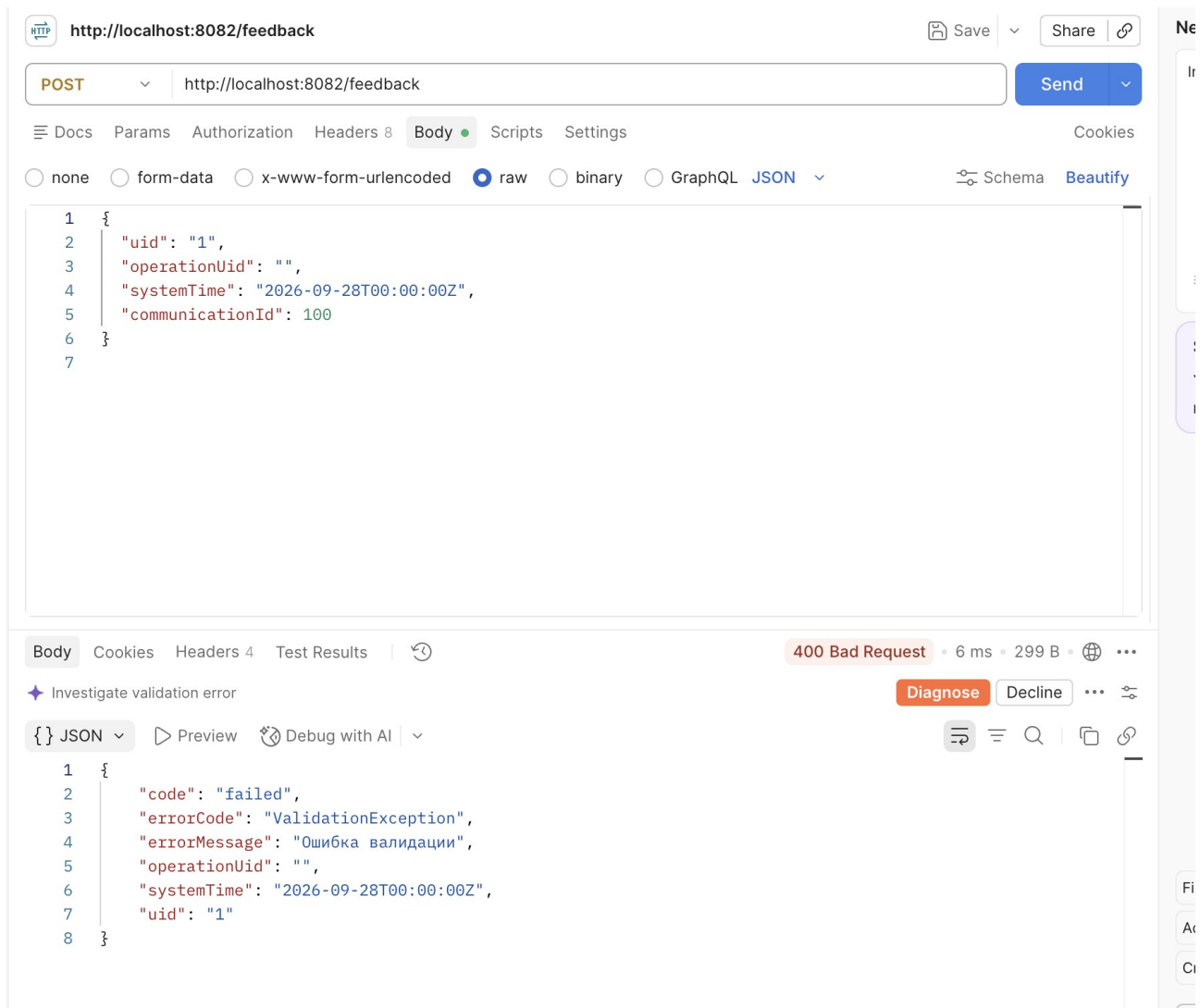


Рисунок 13. Проверка пустого значения operationUid в Postman

3.1.4. Проверка длины operationUid более 32 символов

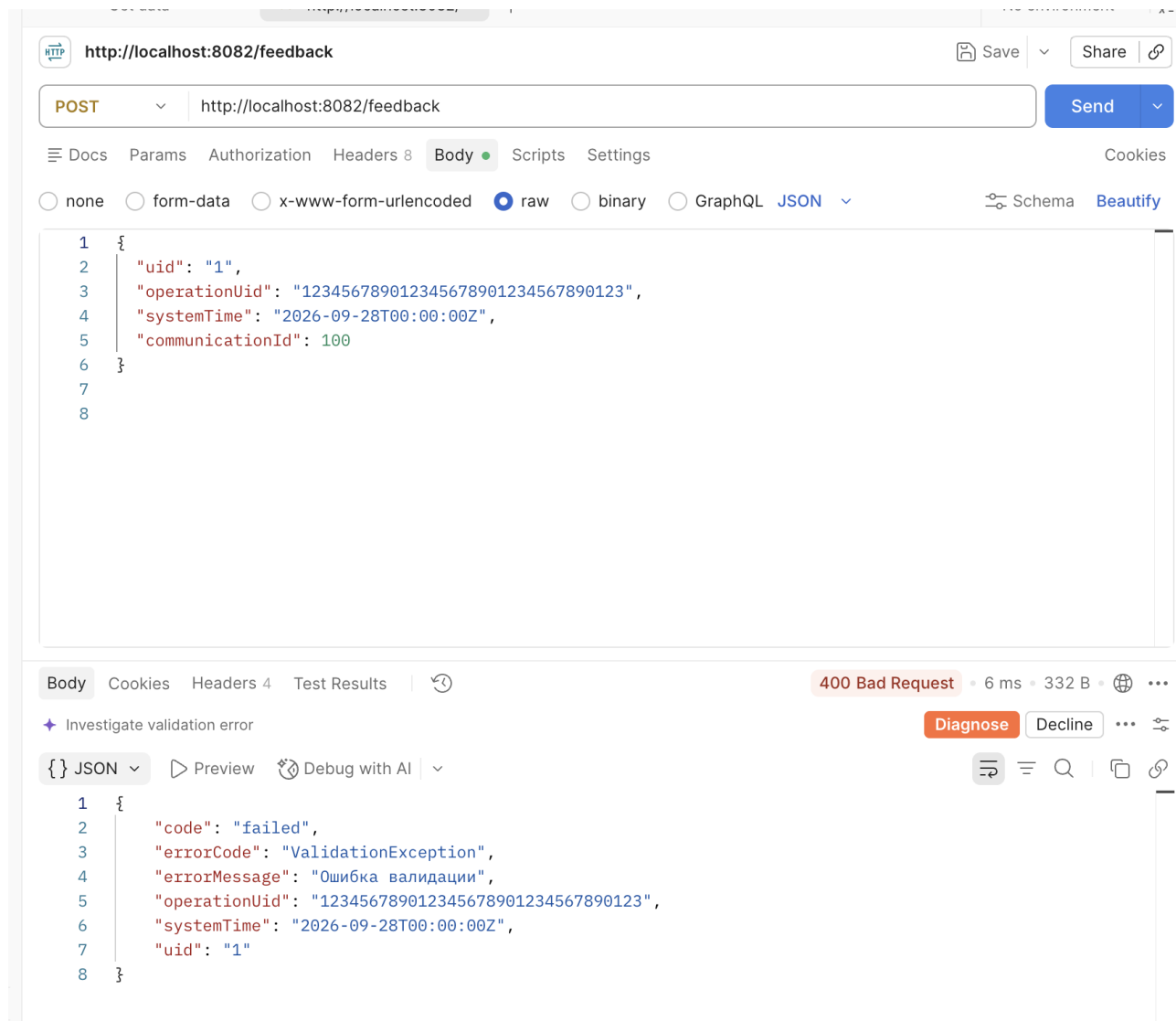


Рисунок 14. Проверка длины operationUid более 32 символов в Postman

3.1.5. Проверка обязательности systemTime

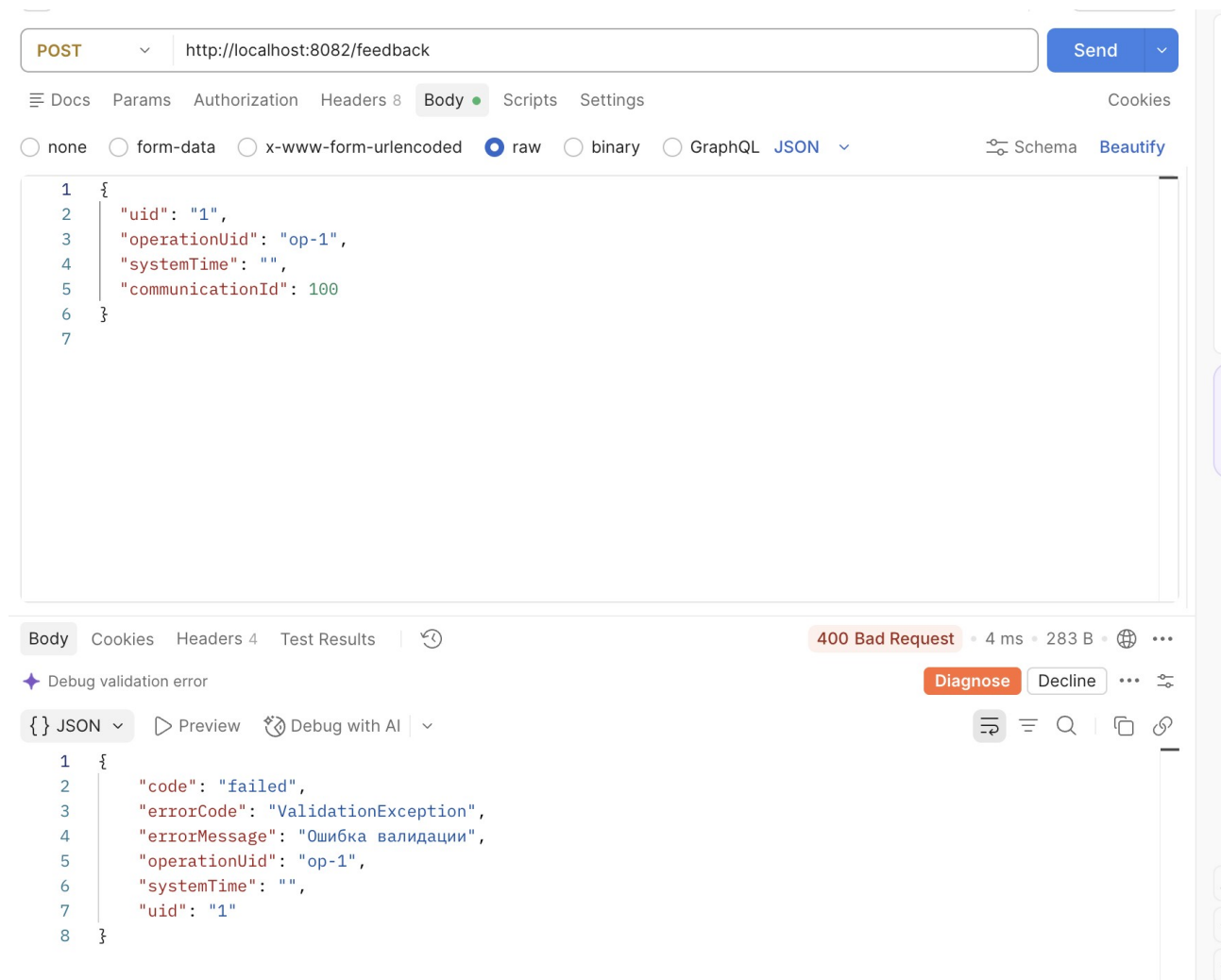


Рисунок 15. Проверка обязательности systemTime в Postman

3.1.6. Проверка нижней границы communicationId

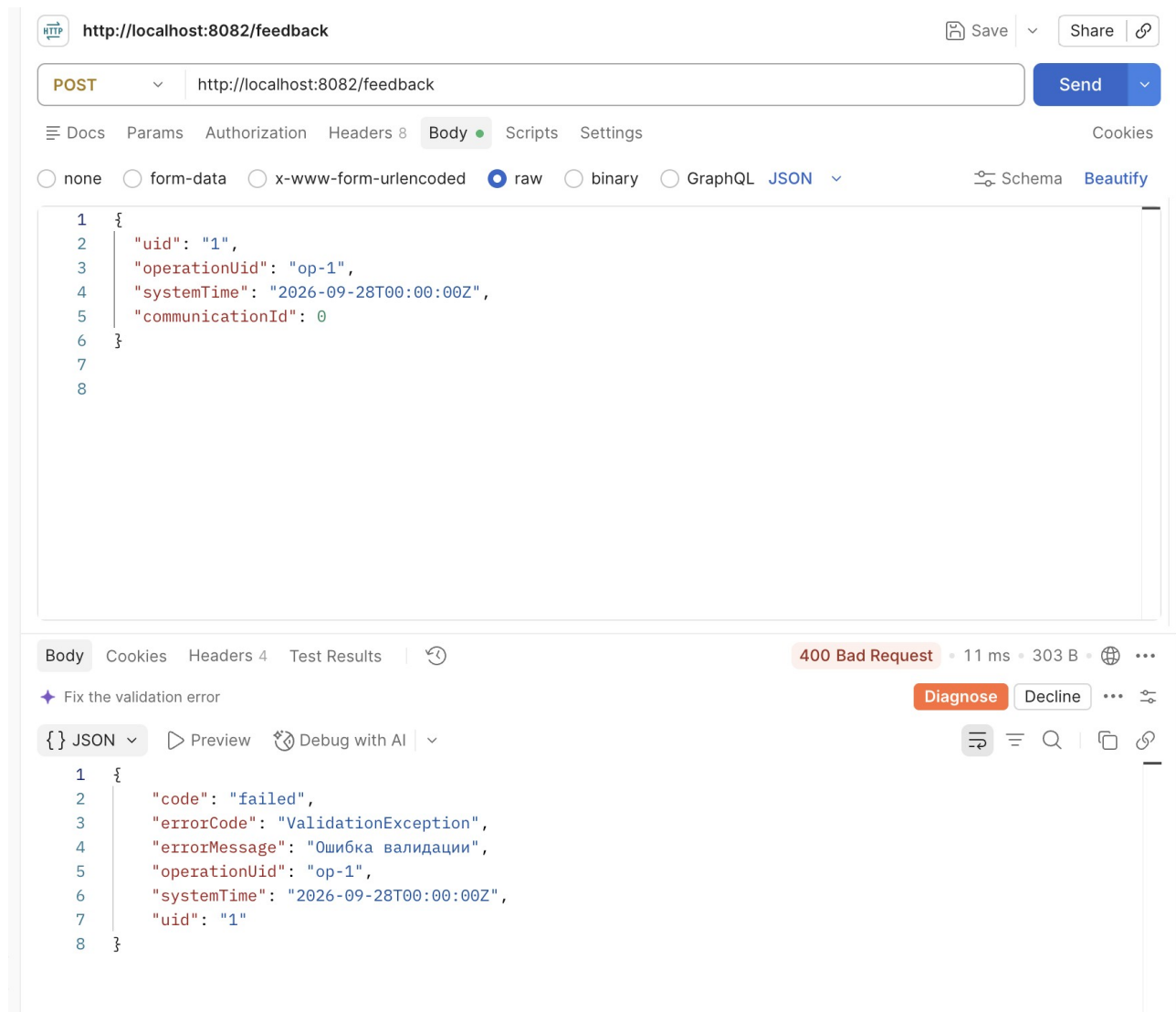


Рисунок 16. Проверка нижней границы communicationId в Postman

3.1.7. Проверка верхней границы communicationId

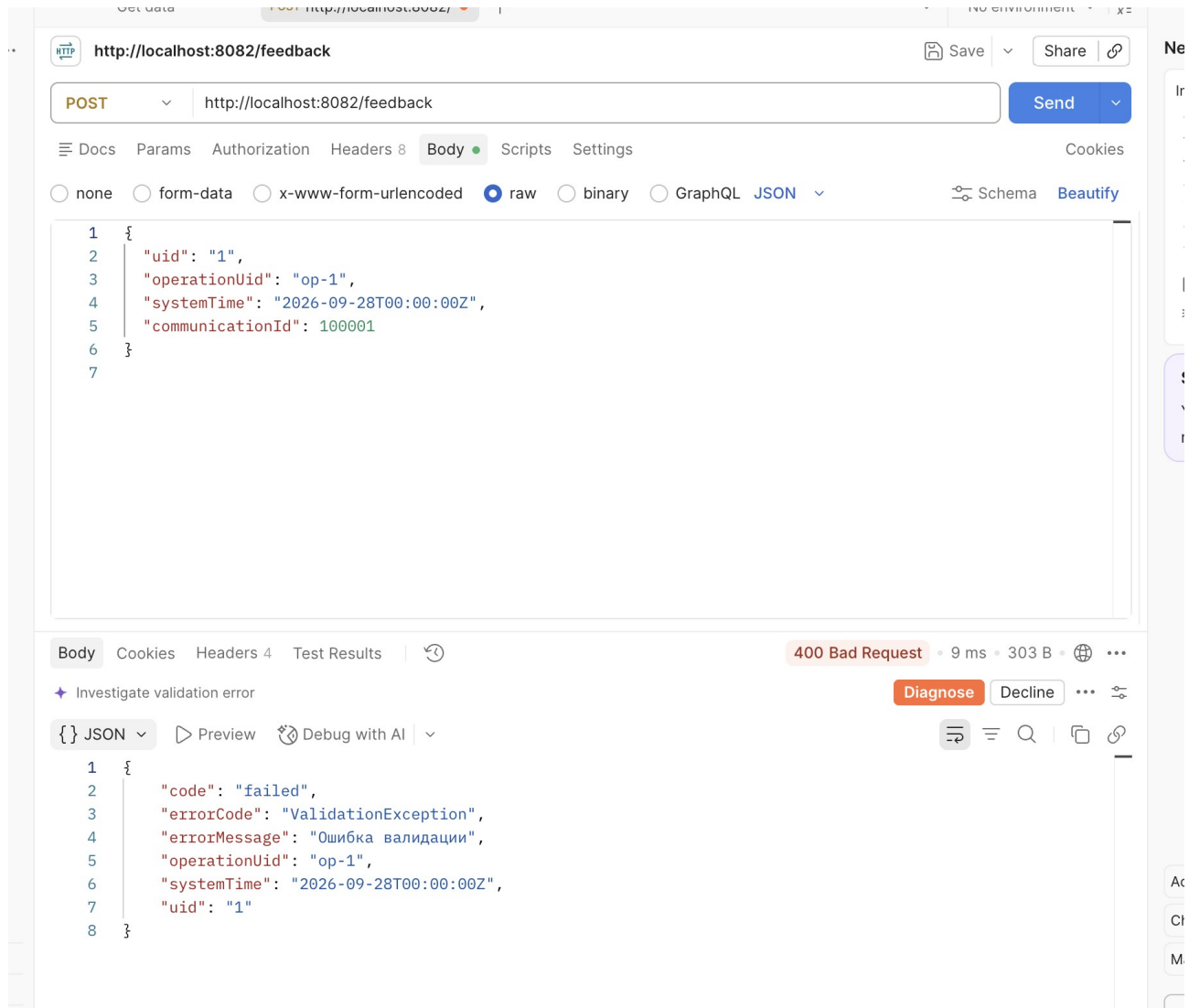


Рисунок 17. Проверка верхней границы communicationId в Postman

3.1.8. Проверка нескольких ошибок валидации одновременно

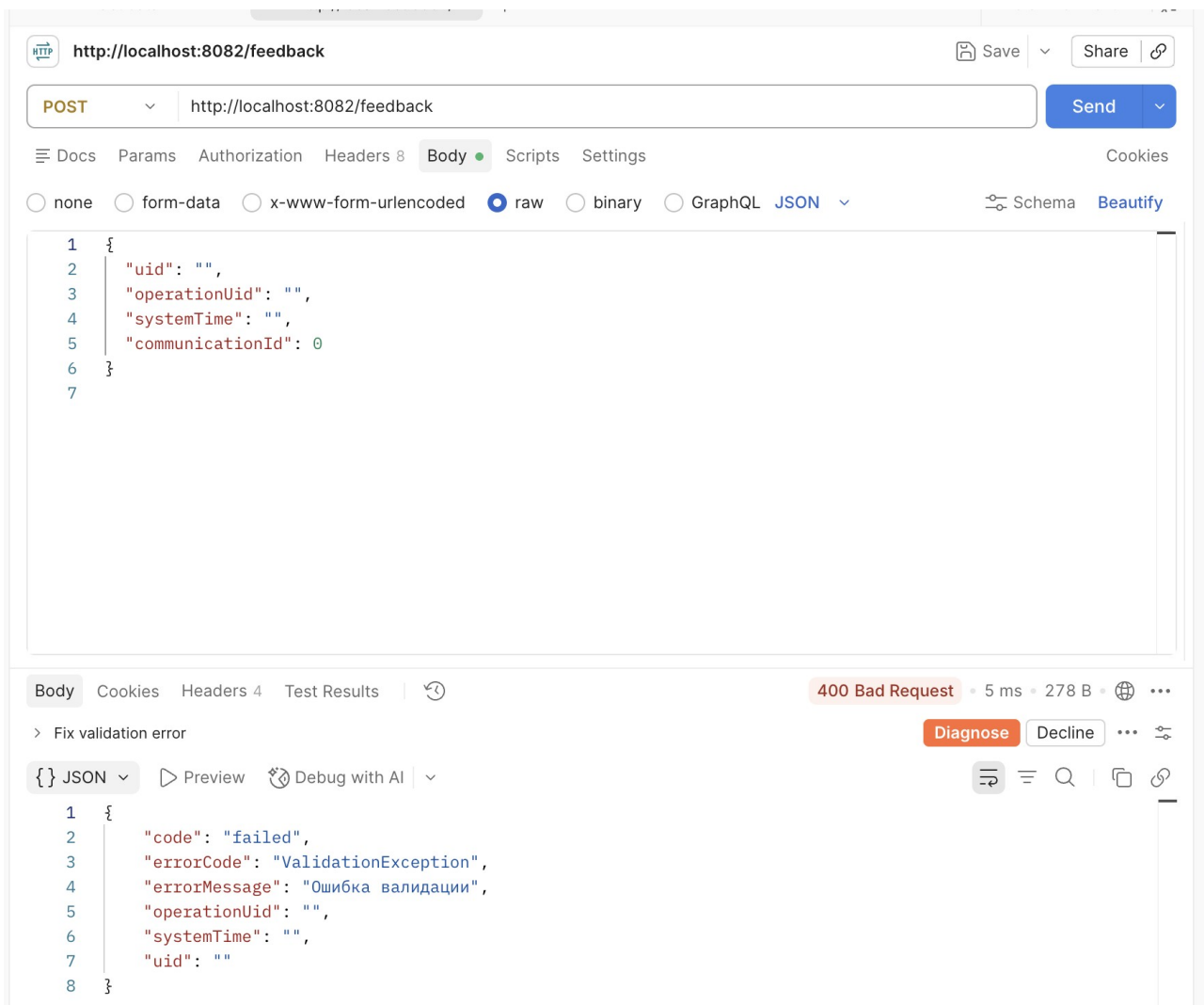


Рисунок 18. Проверка нескольких ошибок валидации одновременно в Postman

3.2. Исключение `UnsupportedCodeException`

Создается собственное исключение `UnsupportedCodeException`. При `uid = 123` сервис должен вернуть ответ с соответствующим кодом ошибки.

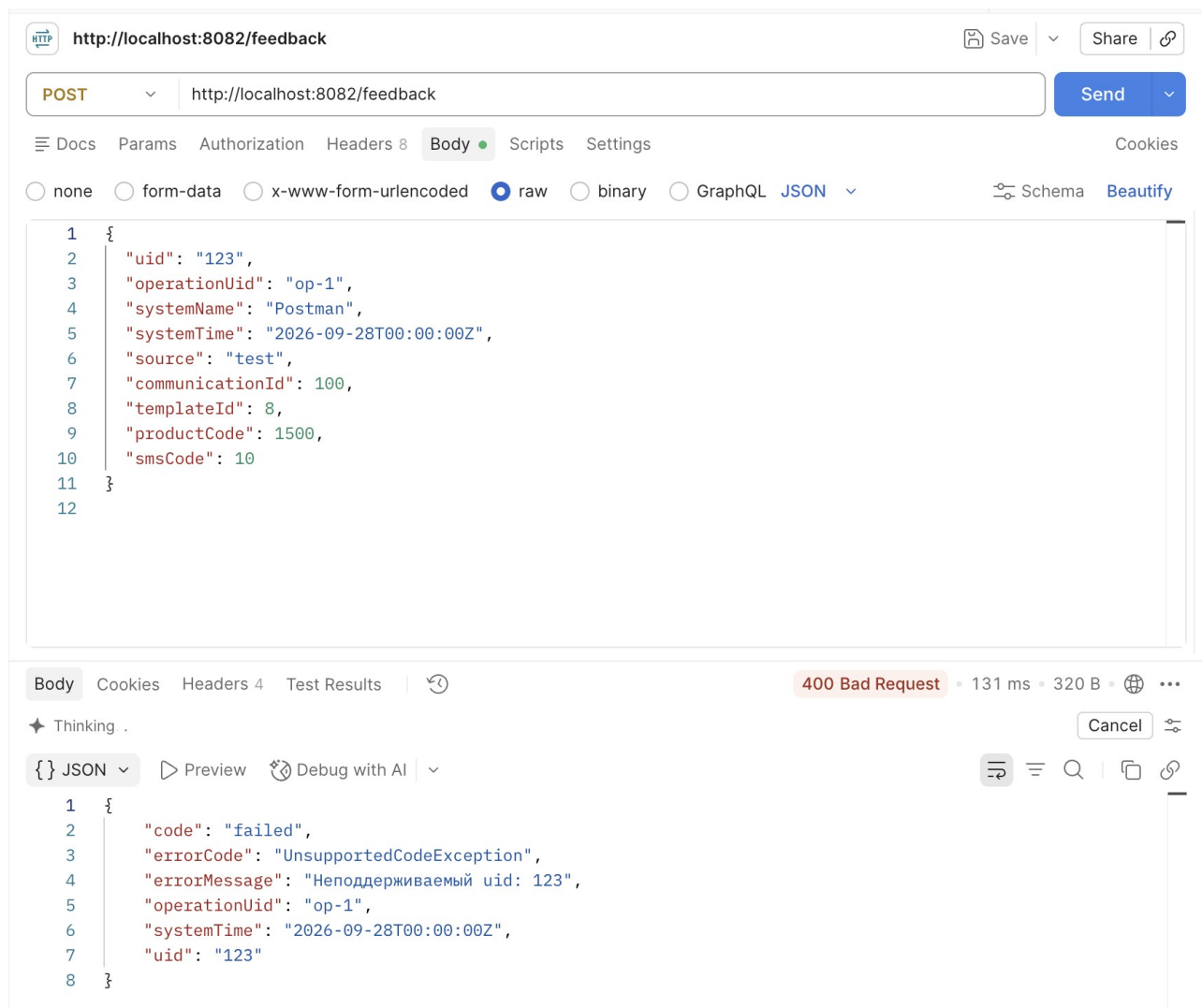


Рисунок 19. Тестирование `uid = 123` в Postman

4. Ответы на вопросы

5. Что означает аннотация `@Service`? Аннотация `@Service` обозначает класс как компонент сервисного слоя Spring. Она позволяет Spring автоматически обнаруживать класс, создавать его объект (Bean) и внедрять в другие компоненты через механизм `Dependency Injection`. Обычно используется для классов, содержащих бизнес-логику приложения.

5. Вывод

В ходе лабораторной работы был разработан REST-сервис на Spring Boot, реализован POST-запрос и обработка JSON-сообщений. Созданы классы Request и Response, добавлена валидация входящих данных и обработка исключений ValidationFailedException и UnsupportedOperationException. Работа сервиса проверена с помощью Postman, включая тестирование корректных запросов и различных ошибок валидации. В результате были закреплены навыки разработки REST API, работы с аннотациями Spring Boot, валидации данных и обработки исключений.